Training

Document 5 of 6 — Building a GPT from Scratch

This document covers everything that happens between having a model and having a trained model: the data pipeline, the optimizer, the learning rate schedule, and the training loop. Each component has a clear mathematical motivation.

1 Character-Level Tokenization

Before the model can process text, we need to convert characters to integers. TinyShakespeare contains approximately 1,115,394 characters drawn from a vocabulary of 65 unique characters (lowercase and uppercase letters, digits, punctuation, spaces, and newlines).

Our tokenizer (in `src/tokenizer.py`) is the simplest possible: sort the unique characters, assign each an integer index, and store two lookup dictionaries.

File: `src/tokenizer.py`

```python
class CharTokenizer:
    """Character-level tokenizer (Section 1).

    encode: string -> list of ints (one per character).
    decode: list of ints -> string. Vocab = sorted unique chars.
    """

    def __init__(self, text: str):
        chars = sorted(set(text))
        self.char_to_idx = {ch: i for i, ch in enumerate(chars)}
        self.idx_to_char = {i: ch for i, ch in enumerate(chars)}

    @property
    def vocab_size(self) -> int:
        return len(self.char_to_idx)

    def encode(self, text: str) -> list[int]:
        return [self.char_to_idx[ch] for ch in text]

    def decode(self, tokens: list[int]) -> str:
        return "".join(self.idx_to_char[t] for t in tokens)
```

Encoding is a character-by-character lookup. Decoding is the reverse. That is the entire tokenizer.

1.1 Byte Pair Encoding (BPE)

Production language models do not tokenize character by character. They use **Byte Pair Encoding (BPE)** (Sennrich et al., 2016), a subword tokenization algorithm that finds a vocabulary of variable-length tokens by greedily merging frequent character pairs. The result sits between character-level and word-level tokenization: common words become single tokens, rare words are split into recognizable pieces, and any string can be represented.

1.1.1 The training algorithm

BPE learns its vocabulary from a training corpus. The algorithm is simple: start with a vocabulary of individual characters, then repeatedly merge the most frequent adjacent pair.

1. **Initialize.** Split every word in the corpus into a sequence of characters. The initial vocabulary is the set of all unique characters.
2. **Count pairs.** For every adjacent pair of tokens in every word, count how many times that pair appears across the entire corpus.
3. **Merge the top pair.** Find the pair with the highest count. Create a new token by concatenating the two tokens in the pair. Replace every occurrence of that pair in the corpus with the new token. Add the new token to the vocabulary.
4. **Repeat.** Go back to step 2. Stop after a predetermined number of merges (this controls the final vocabulary size).

Pseudocode:

Conceptual — illustrating the idea

```python
def train_bpe(corpus: list[list[str]], num_merges: int) ->
    list[tuple[str, str]]:
    """Learn BPE merge rules from a tokenized corpus.

    corpus: list of words, each word is a list of character tokens.
            e.g. [["l","o","w"], ["l","o","w","e","r"], ...]
    Returns: ordered list of merge rules [(pair_a, pair_b), ...].
    """
    merges = []

    for i in range(num_merges):
        # Count every adjacent pair across all words
        pair_counts = {}
        for word in corpus:
            for j in range(len(word) - 1):
                pair = (word[j], word[j + 1])
                pair_counts[pair] = pair_counts.get(pair, 0) + 1

        if not pair_counts:
            break

        # Find the most frequent pair
        best_pair = max(pair_counts, key=pair_counts.get)
        merges.append(best_pair)

        # Merge: replace every occurrence of best_pair with a new token
        new_token = best_pair[0] + best_pair[1]
        for word in corpus:
            k = 0
            while k < len(word) - 1:
                if (word[k], word[k + 1]) == best_pair:
                    word[k:k + 2] = [new_token]
                else:
                    k += 1

    return merges
```

1.1.2 Worked example

Suppose the training corpus contains these words with their frequencies:

Word (as characters)	Frequency
l o w	5
l o w e r	2
n e w e s t	6
w i d e s t	3

Initial vocabulary: $\{\texttt{d}, \texttt{e}, \texttt{i}, \texttt{l}, \texttt{n}, \texttt{o}, \texttt{r}, \texttt{s}, \texttt{t}, \texttt{w}\}$ (10 tokens).

Merge 1. Count all adjacent pairs (weighted by word frequency):

$$\texttt{(e,s)} = 9 \quad \texttt{(s,t)} = 9 \quad \texttt{(l,o)} = 7 \quad \texttt{(o,w)} = 7 \quad \texttt{(n,e)} = 6 \quad \texttt{(e,w)} = 6 \quad \ldots$$

Tie between `(e,s)` and `(s,t)` at 9. Pick `(e,s)`. Merge into `es`. The corpus becomes:

$$\texttt{l o w}\ (5), \quad \texttt{l o w e r}\ (2), \quad \texttt{n e w es t}\ (6), \quad \texttt{w i d es t}\ (3)$$

Vocabulary: 11 tokens (added `es`).

Merge 2. Now `(es,t)` appears $6 + 3 = 9$ times — highest. Merge into `est`:

$$\texttt{l o w}\ (5), \quad \texttt{l o w e r}\ (2), \quad \texttt{n e w est}\ (6), \quad \texttt{w i d est}\ (3)$$

Merge 3. `(l,o)` appears 7 times. Merge into `lo`:

$$\texttt{lo w}\ (5), \quad \texttt{lo w e r}\ (2), \quad \texttt{n e w est}\ (6), \quad \texttt{w i d est}\ (3)$$

After more merges, common subwords like `low`, `new`, `est` become single tokens. Rare combinations stay as smaller pieces.

1.1.3 Encoding new text

Once the merge rules are learned, encoding a new string applies the same merges in order:

Conceptual — illustrating the idea

```python
def encode_bpe(text: str, merges: list[tuple[str, str]]) -> list[str]:
    """Encode a string using learned BPE merge rules.

    Start with individual characters, then apply each merge rule
    in the order it was learned.
    """
    tokens = list(text)          # start as characters
    for pair in merges:
        new_token = pair[0] + pair[1]
        i = 0
        while i < len(tokens) - 1:
            if (tokens[i], tokens[i + 1]) == pair:
                tokens[i:i + 2] = [new_token]
            else:
                i += 1
    return tokens
```

For example, with the merges above, `"lowest"` encodes as `["low", "est"]` — two tokens instead of six characters.

1.1.4 Byte-level BPE (GPT-2)

GPT-2 uses a variant called **byte-level BPE**. Instead of starting with Unicode characters, the base vocabulary is all 256 individual bytes (`0x00` through `0xFF`). This guarantees that any input — including Unicode, emojis, or arbitrary binary data — can be tokenized without an "unknown token" fallback. GPT-2 performs approximately 50,000 merges, yielding a vocabulary of about 50,257 tokens.

1.1.5 Why we use character-level instead

Property	Character-level	BPE (subword)
Vocab size	65	30K – 50K
Sequence length	Long (1 char = 1 token)	Short (~4x fewer)
Tokenizer complexity	Two dictionaries	Merge table + algorithm
Info per token	1 character	~1 word or subword

BPE produces shorter sequences, which means the model's fixed context window covers more text. It also lets the model reason at word-level granularity rather than character-level. The trade-off is tokenizer complexity and a much larger embedding table.

We use character-level tokenization because it makes the full pipeline transparent. There are no merge rules, no special tokens, no tokenizer artifacts. Every step from raw text to model input is visible.

1.2 Label Smoothing

The original "Attention Is All You Need" paper trains with **label smoothing** ($\epsilon = 0.1$). Instead of a hard one-hot target, the correct class gets probability $1 - \epsilon + \epsilon/K$ and every other class gets ϵ/K:

$$\text{target}_i = \begin{cases} 1 - \epsilon + \epsilon/K & \text{if } i = \text{correct class} \\ \epsilon/K & \text{otherwise} \end{cases}$$

This prevents the model from pushing logits toward $\pm\infty$ to achieve a perfect one-hot output. Once the correct token reaches ~90% probability, there is no further gradient signal — the model is discouraged from over-fitting to individual training examples. We do not use label smoothing in BabyGPT; at this scale the benefit is minimal, and standard cross-entropy keeps the code simpler.

2 Building the Dataset

The data pipeline is in `src/dataset.py`. The key steps:

These snippets show the logic inside `src/dataset.py`, which you should create next.

1. Download and load. First, download the TinyShakespeare text file into the `data/` directory:

Shell — run in your terminal

```
mkdir -p data
URL=https://raw.githubusercontent.com/karpathy/char-rnn
URL=$URL/master/data/tinyshakespeare/input.txt
wget -O data/tinyshakespeare.txt $URL
```

Then read the file, encode all characters to integers, and store the result as a single long tensor.

Conceptual — illustrating the idea

```
text = DATA_FILE.read_text()                                    # ~1.1M
    characters
tokenizer = CharTokenizer(text)
data = torch.tensor(tokenizer.encode(text), dtype=torch.long)   # shape:
    (1115394,)
```

2. Train/val split. The first 90% of the data is training, the last 10% is validation. We split sequentially (not randomly) because the data is a continuous text — random splitting would leak context across the boundary.

Conceptual — illustrating the idea

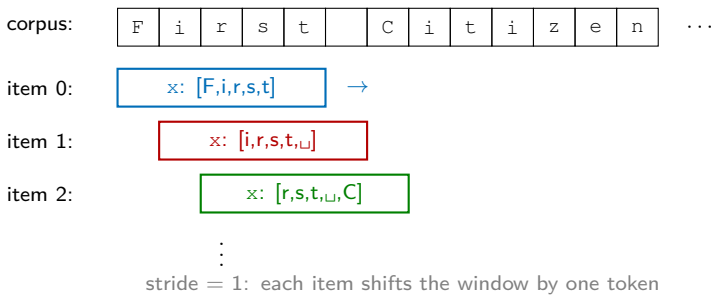
```
split = int(0.9 * len(data))
train_data = data[:split]    # ~1,003,854 tokens
val_data = data[split:]      # ~111,540 tokens
```

3. Create training examples. Each training example is a contiguous slice of `block_size + 1 = 257` tokens. The input `x` is the first 256 tokens; the target `y` is the same window shifted right by one:

Conceptual — illustrating the idea

```
def __getitem__(self, idx):
    chunk = self.data[idx : idx + self.block_size + 1]  # 257 tokens
    x = chunk[:-1]    # tokens 0..255 -- the input
    y = chunk[1:]      # tokens 1..256 -- the target
    return x, y
```

The dataset generates examples by **striding with step size 1**: item 0 starts at position 0, item 1 starts at position 1, item 2 at position 2, and so on. Consecutive items overlap by 256 of 257 tokens. This means almost every token in the corpus appears as a target in many different contexts.



The total number of items is `len(data) - block_size`. For the training split: 1,003,854 − 256 = 1,003,598 overlapping windows. The `DataLoader` shuffles these and draws batches of 64.

Every position in `x` provides a training signal. Position 0 must predict position 1 (using only token 0 as context). Position 255 must predict position 256 (using all 256 tokens of context). A single `(x, y)` pair gives us 256 next-token prediction examples.

4. DataLoader. Standard PyTorch DataLoader wraps the dataset:

Conceptual — illustrating the idea

```
train_loader = DataLoader(
    train_dataset,
    batch_size=64,
    shuffle=True,           # random order for training
    pin_memory=True,        # faster CPU->GPU transfer
    drop_last=True,         # drop incomplete final batch
)
```

Each batch contains 64 sequences of length 256, giving $64 \times 256 = 16{,}384$ tokens per batch.

2.1 Creating the Data Pipeline

Create the tokenizer and dataset files. The tokenizer was shown above; here is the full dataset module:

File: `src/dataset.py`

```
"""TinyShakespeare dataset: tokenize and create DataLoaders."""

from pathlib import Path

import torch
from torch.utils.data import Dataset, DataLoader

from src.tokenizer import CharTokenizer
from src.config import GPTConfig

DATA_FILE = Path(__file__).resolve().parent.parent / "data" /
    "tinyshakespeare.txt"


class ShakespeareDataset(Dataset):
    """Yields (x, y) pairs where x = chunk[:-1] and y = chunk[1:].

    Each sample is a contiguous slice of block_size+1 characters from the
    encoded text. The input x is the first block_size tokens; the target
        y
    is the same window shifted right by one position. Every position in x
    provides one next-token prediction training example.
    """

    def __init__(self, data: torch.Tensor, block_size: int):
        self.data = data
        self.block_size = block_size

    def __len__(self) -> int:
        return len(self.data) - self.block_size

    def __getitem__(self, idx: int) -> tuple[torch.Tensor, torch.Tensor]:
        chunk = self.data[idx : idx + self.block_size + 1]
        x = chunk[:-1]  # (block_size,)
        y = chunk[1:]   # (block_size,)
        return x, y


def get_dataloaders(
    config: GPTConfig,
```

```python
    batch_size: int = 64,
    num_workers: int = 0,
) -> tuple[DataLoader, DataLoader, CharTokenizer]:
    """Read data, build tokenizer, return (train_loader, val_loader,
        tokenizer)."""
    text = DATA_FILE.read_text()
    tokenizer = CharTokenizer(text)

    # Encode entire text as a single long tensor
    data = torch.tensor(tokenizer.encode(text), dtype=torch.long)

    # 90/10 train/val split
    split = int(0.9 * len(data))
    train_data = data[:split]
    val_data = data[split:]

    train_dataset = ShakespeareDataset(train_data, config.block_size)
    val_dataset = ShakespeareDataset(val_data, config.block_size)

    train_loader = DataLoader(
        train_dataset,
        batch_size=batch_size,
        shuffle=True,
        num_workers=num_workers,
        pin_memory=True,
        drop_last=True,
    )
    val_loader = DataLoader(
        val_dataset,
        batch_size=batch_size,
        shuffle=False,
        num_workers=num_workers,
        pin_memory=True,
        drop_last=True,
    )

    return train_loader, val_loader, tokenizer


if __name__ == "__main__":
    text = DATA_FILE.read_text()
    tok = CharTokenizer(text)
    print(f"Characters: {len(text):,}")
    print(f"Vocab size: {tok.vocab_size}")
    print(f"Vocab: {''.join(sorted(tok.char_to_idx.keys()))!r}")

    data = torch.tensor(tok.encode(text), dtype=torch.long)
    split = int(0.9 * len(data))
    print(f"Train tokens: {split:,}")
    print(f"Val tokens:   {len(data) - split:,}")
```

Shell — run in your terminal

```shell
uv run python -m src.dataset
```

Checkpoint: You should see: `Characters: 1,115,394`, `Vocab size: 65`, `Train tokens: 1,003,854`, `Val tokens: 111,540`.

Inspecting a training sample. You can decode a sample back to text to see exactly what the model trains on:

Demo — try in a REPL

```
from src.config import GPTConfig
from src.dataset import get_dataloaders

config = GPTConfig()
train_loader, val_loader, tok = get_dataloaders(config)

x, y = next(iter(train_loader))          # one batch: (64, 256) each
print("Input: ", tok.decode(x[0, :40].tolist()))
print("Target:", tok.decode(y[0, :40].tolist()))

# Output:
# Input:  First Citizen:\nBefore we proceed any
# Target: irst Citizen:\nBefore we proceed any f
```

The target is the input shifted by one character — at every position, the model learns to predict the next character.

Tip: Why does a shifted target teach next-token prediction — and how does weight tying fit in?
Two architectural constraints make this work. **(1) The causal mask** ensures position t only attends to tokens $x_0, \ldots, x_t$ — it cannot see x_{t+1} or beyond. So the hidden vector at position t is a learned summary of the past, not a copy of the future. **(2) The cross-entropy loss** compares position t's output against the target x_{t+1}. Gradients flow backward and reshape the model so that each position's hidden representation becomes useful for predicting what comes next — because that is the only thing the loss ever rewards.
Now recall that `lm_head` shares weights with `wte` (the token embedding matrix, shape 65×384). The logit for character i at position t is simply:

$$\text{logit}[t, i] = \text{dot}(\text{hidden}_t,\ \text{embedding}_i)$$

This is a similarity search: "which token's embedding is the hidden state closest to?" The hidden state at position t *starts* as the embedding of x_t (plus the positional embedding), but 6 transformer blocks then transform it — attention mixes in context from all positions $0 \ldots t$, and feed-forward layers apply nonlinear transformations. By the time the vector exits the final LayerNorm, it is no longer "the embedding of x_t."
The training loss forces the transformer blocks to push that hidden vector **toward the embedding of** x_{t+1}. Think of the embedding space as a shared room where every character has a fixed address: the transformer blocks move the hidden state from near x_t's address toward x_{t+1}'s address, and the tied output head asks "whose address are you closest to now?" Weight tying constrains input and output to share the same geometry — tokens that appear in similar contexts get similar embeddings *and* are predicted in similar contexts. This is a useful inductive bias, not a limitation.

3 The Optimizer: AdamW

AdamW (Loshchilov & Hutter, 2019) is the standard optimizer for transformer training. It combines two ideas:

1. **Adaptive learning rates.** Each parameter gets its own effective learning rate based on the history of its gradients. Parameters with consistently large gradients receive smaller updates; parameters with small gradients receive larger updates. This is the Adam part.
2. **Decoupled weight decay.** Weight matrices are shrunk toward zero by a fixed fraction each step ($\alpha\lambda$), independent of the gradient history. This regularization prevents any single weight from growing too large. The "decoupled" means the decay is applied separately from the adaptive gradient step, which is the key difference from naively adding L2 regularization to Adam.

The update rule:

$$\theta_{t+1} = \theta_t - \alpha \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} - \alpha \cdot \lambda \cdot \theta_t$$

where $\hat{m}_t$ is the bias-corrected moving average of gradients (momentum), $\hat{v}_t$ is the bias-corrected moving average of squared gradients (adaptive scaling), and λ is the weight decay coefficient.

Not all parameters should be decayed. Weight matrices (2D tensors) get decay; biases and LayerNorm parameters (1D tensors) do not — they are offsets and scales that should be free to take whatever value is correct.

Parameter	Value
Learning rate (α)	1×10^{-3}
β_1 (momentum)	0.9
β_2 (adaptive scaling)	0.99
Weight decay (λ)	0.1 (2D params only)

4 Learning Rate Schedule: Cosine Annealing with Warmup

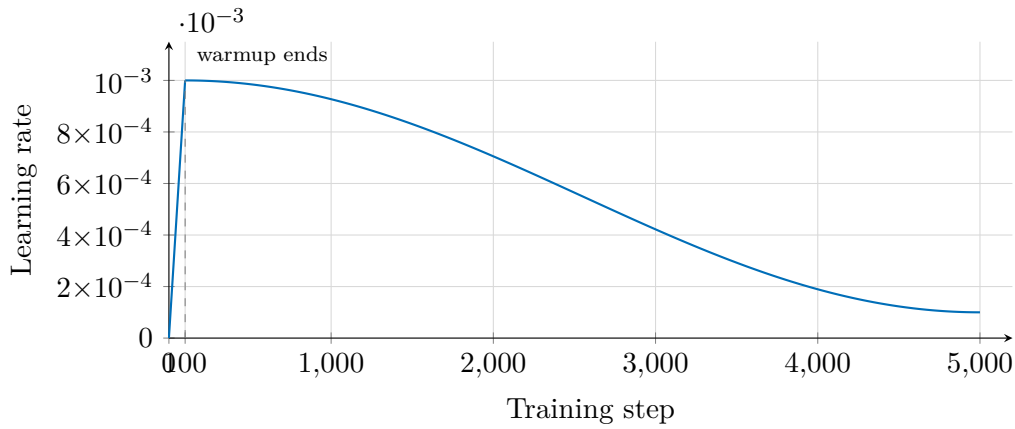
The learning rate follows a two-phase schedule: linear warmup, then cosine decay.

Warmup (steps 0–100): $\text{lr}(t) = \text{max_lr} \cdot t \,/\, \text{warmup_iters}$. The learning rate ramps linearly from 0 to the maximum. This gives AdamW's moment estimates time to stabilize before the optimizer takes large steps.

Cosine decay (steps 100–5000):

$$\text{lr}(t) = \text{min_lr} + \tfrac{1}{2}(\text{max_lr} - \text{min_lr})\big(1 + \cos(\pi \cdot \text{progress})\big)$$

where $\text{progress} = (t - \text{warmup_iters})/(\text{decay_iters} - \text{warmup_iters})$ goes from 0 to 1. The cosine curve spends more time near the maximum early on (when the model is making large progress) and near the minimum late in training (when fine adjustments matter).



5 The Training Loop

The training loop in `src/train.py` follows the standard PyTorch pattern, with mixed precision and gradient clipping. Here is the core of the loop, annotated step by step:

Conceptual — illustrating the idea

From the main training loop in `src/train.py`.

```
for step in range(max_iters):
    # 1. Set learning rate for this step
    lr = get_lr(step)
    for param_group in optimizer.param_groups:
        param_group["lr"] = lr

    # 2. Get a batch from the DataLoader, move to GPU
    x, y = next(train_iter)          # x: (64, 256), y: (64, 256)
    x, y = x.to(device), y.to(device)

    # 3. Forward pass with mixed precision
    with torch.amp.autocast("cuda", dtype=torch.float16,
        enabled=use_amp):
        _, loss = model(x, y)

    # 4. Backward pass with loss scaling
    scaler.scale(loss).backward()

    # 5. Unscale gradients for correct norm computation
    scaler.unscale_(optimizer)

    # 6. Gradient clipping
    torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)

    # 7. Optimizer step and scaler update
    scaler.step(optimizer)
    scaler.update()

    # 8. Clear gradients
    optimizer.zero_grad(set_to_none=True)
```

Let us walk through each piece.

5.1 Mixed Precision Training

Modern GPUs have specialized hardware (tensor cores) that operate much faster on 16-bit floating point than on 32-bit. Mixed precision training exploits this:

- **Forward and backward pass:** Run in float16. This uses roughly half the memory and is faster on tensor cores.
- **Optimizer state and parameter updates:** Kept in float32. The optimizer needs full precision because it computes small corrections (learning rate times gradient divided by moment estimate) that would lose significant digits in float16.

The `torch.amp.autocast` context manager automatically casts operations to float16 where it is safe and leaves precision-sensitive operations (like softmax, loss computation) in float32.

The GradScaler: Float16 has a limited range — the smallest representable positive number is about 6×10^{-8}. Gradients for some parameters can be smaller than this, which means they round to zero (underflow). The GradScaler multiplies the loss by a large factor before `backward()`, which scales all gradients up into the representable range. Before the optimizer step, `scaler.unscale_()` divides the gradients back to their true values. The scale factor is adjusted dynamically: if gradients contain inf/nan (overflow), the scaler reduces the factor and skips the optimizer step.

5.2 Gradient Clipping

Conceptual — illustrating the idea
From the main training loop in `src/train.py`.

```
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
```

This computes the global norm of all gradients:

$$\|g\| = \sqrt{\sum_i \|g_i\|^2}$$

where the sum is over all parameter tensors. If $\|g\| > \text{max_norm}$, every gradient is scaled down by $\text{max_norm}/\|g\|$ so that the global norm becomes exactly `max_norm`.

Why this matters: occasionally, a batch produces anomalously large gradients (an "exploding gradient"). Without clipping, a single bad batch can make a destructive parameter update that ruins training progress. Gradient clipping bounds the worst-case step size while leaving normal-sized gradients unchanged.

Note: `scaler.unscale_()` must be called before gradient clipping. Otherwise you would be clipping the scaled gradients, and the effective max norm would vary with the current scale factor.

5.3 Clearing Gradients

Conceptual — illustrating the idea
From the main training loop in `src/train.py`.

```
optimizer.zero_grad(set_to_none=True)
```

PyTorch accumulates gradients by default — calling `.backward()` adds to the existing `.grad` attribute. We need to clear gradients before each step. The `set_to_none=True` argument sets `.grad = None` instead of filling with zeros. This is slightly faster (avoids a memset) and uses no memory until the next backward pass populates the gradients.

5.4 Periodic Evaluation

Every 250 steps, the training loop evaluates the model on both training and validation data by averaging the loss over 200 batches:

Conceptual — illustrating the idea
From the main training loop in `src/train.py`.

```
@torch.no_grad()
def estimate_loss(model, train_loader, val_loader, device,
    eval_iters=200):
    model.eval()
    # ... average loss over eval_iters batches for each split ...
    model.train()
    return {"train": train_loss, "val": val_loss}
```

`model.eval()` disables dropout (all tokens are kept, but activations are not scaled — PyTorch handles this automatically). `@torch.no_grad()` disables gradient tracking, saving memory during evaluation. After evaluation, `model.train()` re-enables dropout.

5.5 Checkpointing

When the validation loss improves, the model state is saved:

Conceptual — illustrating the idea
From the main training loop in `src/train.py`.

```
checkpoint = {
    "model": model.state_dict(),
    "config": config,
    "step": step,
    "val_loss": best_val_loss,
    "vocab": tokenizer.char_to_idx,
}
torch.save(checkpoint, CHECKPOINT_DIR / "best.pt")
```

The checkpoint includes everything needed to reconstruct the model: the weights, the architecture config, and the vocabulary mapping. This is what `src/generate.py` loads to produce text.

5.6 Running Training

Create `src/train.py` with the complete training script:

File: `src/train.py`

```
"""Training script for BabyGPT on TinyShakespeare."""

import argparse
import math
import os
```

```python
import time
from pathlib import Path

import torch

from src.config import GPTConfig
from src.dataset import get_dataloaders
from src.model import BabyGPT

# ---- Hyperparameters ----
BATCH_SIZE = 64
MAX_ITERS = 5000
EVAL_INTERVAL = 250
EVAL_ITERS = 200

LEARNING_RATE = 1e-3
MIN_LR = 1e-4
WARMUP_ITERS = 100
LR_DECAY_ITERS = 5000

WEIGHT_DECAY = 0.1
BETA1 = 0.9
BETA2 = 0.99
GRAD_CLIP = 1.0

CHECKPOINT_DIR = Path(__file__).resolve().parent.parent / "checkpoints"


def get_lr(step: int) -> float:
    """Cosine annealing with linear warmup.

    - Warmup: linear ramp from 0 to LEARNING_RATE over WARMUP_ITERS
        steps.
    - Cosine decay: smoothly decrease from LEARNING_RATE to MIN_LR.

    Math: lr = min_lr + 0.5 * (max_lr - min_lr) * (1 + cos(pi *
        progress))
    where progress goes from 0 to 1 over the decay phase.
    """
    # Linear warmup
    if step < WARMUP_ITERS:
        return LEARNING_RATE * step / WARMUP_ITERS
    # After decay period, stay at min
    if step > LR_DECAY_ITERS:
        return MIN_LR
    # Cosine decay
    progress = (step - WARMUP_ITERS) / (LR_DECAY_ITERS - WARMUP_ITERS)
    coeff = 0.5 * (1.0 + math.cos(math.pi * progress))
    return MIN_LR + coeff * (LEARNING_RATE - MIN_LR)


def configure_optimizer(model: BabyGPT) -> torch.optim.AdamW:
    """Set up AdamW with separate parameter groups for weight decay.

    Weight matrices get decayed; biases, layer norm params, and
        embeddings don't.
    This is standard practice -- weight decay on biases/norms doesn't
        help and
```

```
    can hurt.
    """
    decay_params = []
    no_decay_params = []

    for name, param in model.named_parameters():
        if not param.requires_grad:
            continue
        # Don't decay 1D parameters (biases, layer norm weights)
        if param.dim() < 2:
            no_decay_params.append(param)
        else:
            decay_params.append(param)

    optim_groups = [
        {"params": decay_params, "weight_decay": WEIGHT_DECAY},
        {"params": no_decay_params, "weight_decay": 0.0},
    ]

    return torch.optim.AdamW(optim_groups, lr=LEARNING_RATE,
       betas=(BETA1, BETA2))


@torch.no_grad()
def estimate_loss(
    model: BabyGPT,
    train_loader: torch.utils.data.DataLoader,
    val_loader: torch.utils.data.DataLoader,
    device: torch.device,
    eval_iters: int = EVAL_ITERS,
) -> dict[str, float]:
    """Estimate train and val loss by averaging over eval_iters
       batches."""
    model.eval()
    out = {}
    for split, loader in [("train", train_loader), ("val", val_loader)]:
        losses = []
        loader_iter = iter(loader)
        for _ in range(eval_iters):
            try:
                x, y = next(loader_iter)
            except StopIteration:
                loader_iter = iter(loader)
                x, y = next(loader_iter)
            x, y = x.to(device), y.to(device)
            _, loss = model(x, y)
            losses.append(loss.item())
        out[split] = sum(losses) / len(losses)
    model.train()
    return out


def train(args: argparse.Namespace) -> None:
    # Device
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    print(f"Using device: {device}")
    if device.type == "cuda":
        print(f"GPU: {torch.cuda.get_device_name()}")
```

```python
# Seed
torch.manual_seed(42)
if device.type == "cuda":
    torch.cuda.manual_seed(42)

# Data
config = GPTConfig()
train_loader, val_loader, tokenizer = get_dataloaders(
    config, batch_size=BATCH_SIZE
)
# Update vocab_size from actual data
config.vocab_size = tokenizer.vocab_size
print(f"Vocab size: {config.vocab_size}")

# Model
model = BabyGPT(config).to(device)
print(f"Model parameters: {model.count_parameters():,}")

# Optimizer
optimizer = configure_optimizer(model)

# Mixed precision
use_amp = device.type == "cuda"
scaler = torch.amp.GradScaler("cuda", enabled=use_amp)
amp_dtype = torch.float16  # RTX 3080 supports bf16 too, but fp16 is
    safe default

# Training loop
CHECKPOINT_DIR.mkdir(parents=True, exist_ok=True)
best_val_loss = float("inf")

train_iter = iter(train_loader)
t0 = time.time()

for step in range(args.max_iters):
    # Set learning rate for this step
    lr = get_lr(step)
    for param_group in optimizer.param_groups:
        param_group["lr"] = lr

    # Get batch (cycle through loader)
    try:
        x, y = next(train_iter)
    except StopIteration:
        train_iter = iter(train_loader)
        x, y = next(train_iter)
    x, y = x.to(device), y.to(device)

    # Forward + backward with mixed precision
    with torch.amp.autocast("cuda", dtype=amp_dtype,
        enabled=use_amp):
        _, loss = model(x, y)

    scaler.scale(loss).backward()

    # Gradient clipping (unscale first for correct norm computation)
    scaler.unscale_(optimizer)
```

```
        torch.nn.utils.clip_grad_norm_(model.parameters(), GRAD_CLIP)

        scaler.step(optimizer)
        scaler.update()
        optimizer.zero_grad(set_to_none=True)

        # Logging and evaluation
        if step % EVAL_INTERVAL == 0 or step == args.max_iters - 1:
            losses = estimate_loss(model, train_loader, val_loader,
                device)
            elapsed = time.time() - t0
            tokens_per_sec = (step + 1) * BATCH_SIZE * config.block_size
                / elapsed
            print(
                f"step {step:5d} | "
                f"train loss {losses['train']:.4f} | "
                f"val loss {losses['val']:.4f} | "
                f"lr {lr:.2e} | "
                f"tok/s {tokens_per_sec:.0f}"
            )

            # Save best checkpoint
            if losses["val"] < best_val_loss:
                best_val_loss = losses["val"]
                checkpoint = {
                    "model": model.state_dict(),
                    "config": config,
                    "step": step,
                    "val_loss": best_val_loss,
                    "vocab": tokenizer.char_to_idx,
                }
                torch.save(checkpoint, CHECKPOINT_DIR / "best.pt")
                print(f"  -> saved checkpoint
                    (val_loss={best_val_loss:.4f})")

    # Save final checkpoint
    checkpoint = {
        "model": model.state_dict(),
        "config": config,
        "step": args.max_iters,
        "val_loss": best_val_loss,
        "vocab": tokenizer.char_to_idx,
    }
    torch.save(checkpoint, CHECKPOINT_DIR / "final.pt")

    total_time = time.time() - t0
    print(f"\nTraining complete in {total_time:.1f}s")
    print(f"Best val loss: {best_val_loss:.4f}")
    print(f"Checkpoints saved to {CHECKPOINT_DIR}")


def main():
    parser = argparse.ArgumentParser(description="Train BabyGPT on
        TinyShakespeare")
    parser.add_argument("--max-iters", type=int, default=MAX_ITERS)
    args = parser.parse_args()
    train(args)
```

```python
if __name__ == "__main__":
    main()
```

Shell — run in your terminal

```shell
uv run python -m src.train
```

Checkpoint: On an RTX 3080 (10 GB VRAM), training takes approximately **13 minutes** for 5000 iterations at ~**103K tokens/sec**, using ~**3.3 GB peak VRAM** with mixed precision. You should see the loss drop from ∼4.17 to a best validation loss around **1.3–1.5**. The best checkpoint is saved to `checkpoints/best.pt`.
Note: our implementation hand-rolls LayerNorm and attention using plain PyTorch operations rather than calling optimized fused kernels (e.g. `F.scaled_dot_product_attention`, `F.layer_norm`). This is intentional — seeing the raw matrix operations is the point of this project — but it means training is slower than an optimized implementation would be. A production codebase using fused kernels and FlashAttention would train the same model significantly faster.

5.7 Hyperparameters Summary

Parameter	Value	Rationale
`batch_size`	64	$64 \times 256 = 16{,}384$ tokens/batch. Fits in 10 GB VRAM.
`max_iters`	5000	≈82M tokens seen. Multiple passes over the dataset.
`learning_rate`	10^{-3}	Relatively high — small models tolerate larger LRs.
`min_lr`	10^{-4}	10x below max. Fine-tunes in the late phase.
`warmup_iters`	100	2% of training. Moment estimates stabilize.
`weight_decay`	0.1	Standard for transformers. Prevents weight growth.
`grad_clip`	1.0	Rarely triggers but prevents catastrophic steps.
`beta1`	0.9	Standard momentum coefficient.
`beta2`	0.99	Faster adaptation than default 0.999.

Next: Document 06 — Generation and Sampling covers autoregressive generation, temperature, top-k sampling, and what to expect from our trained model.